Topic 7: Sinusoidal plus Residual Model

From stochastic approximation to sinusoidal-plus-residual and harmonic-plus-stochastic analysis

Digital Signal Processing for Sound and Music · Topic 7

Xavier Serra · Universitat Pompeu Fabra

Roadmap

1 · Stochastic model

Stochastic signals
Spectral envelopes
Random-phase synthesis

2 · Residual modeling

SpR / HpR
Residual subtraction
Explicit reconstruction error

3 · Stochastic residual

SpS / HpS
Residual envelope
Compact approximation

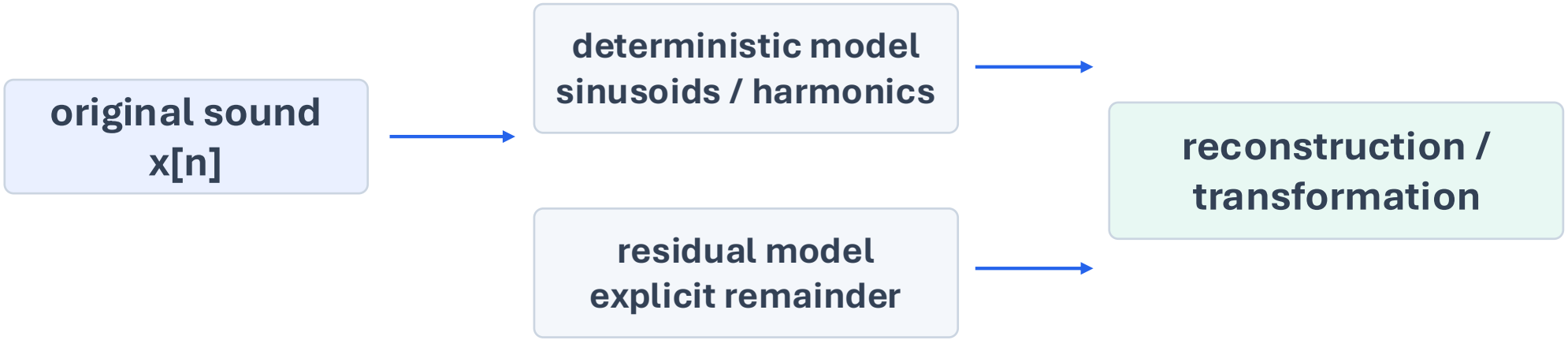
4 · Practice

Examples
Exercise 7
Inspect + listen

Why do we need stochastic or residual components?

Many sounds are not fully explained by stable sinusoids or harmonics.

The deterministic part captures stable peaks. The residual or stochastic part captures what remains: breath, attacks, turbulence, texture, and broadband noise.



Two choices for the residual: keep it explicitly (SpR / HpR), or approximate it statistically (SpS / HpS).

Stochastic signals

A stochastic signal is characterized by stable statistical descriptors, not by one exact sample sequence.

Autocorrelation

$$Z_{xx}[k] = \sum_{n=0}^{N-1} x[n]\, x[n+k] \qquad k = -N+1, \ldots, N-1$$

Power spectral density

$$X_p[k] = \lim_{N \to \infty} |X[k]|^2$$

Discrete Fourier transform

$$X[k] = \frac{1}{N} \sum_{n=0}^{N-1} x[n]\; e^{-j2\pi kn/N} \qquad k = 0, \ldots, N-1$$

Stochastic model

White noise is shaped by a filter or by a short-time spectral envelope.

Time-domain view: white noise shaped by a filter

$$y_{st}[n] = \sum_{k=0}^{N-1} u[k]\, h[n-k]$$

u[n]: white noise h[n]: impulse response of the filter approximating x[n]

Spectral view: magnitude envelope with randomized phases

$$Y_{st,l}[k] = |H_l[k]|\,|U[k]|\, e^{j(\angle H_l[k] + \angle U[k])}$$

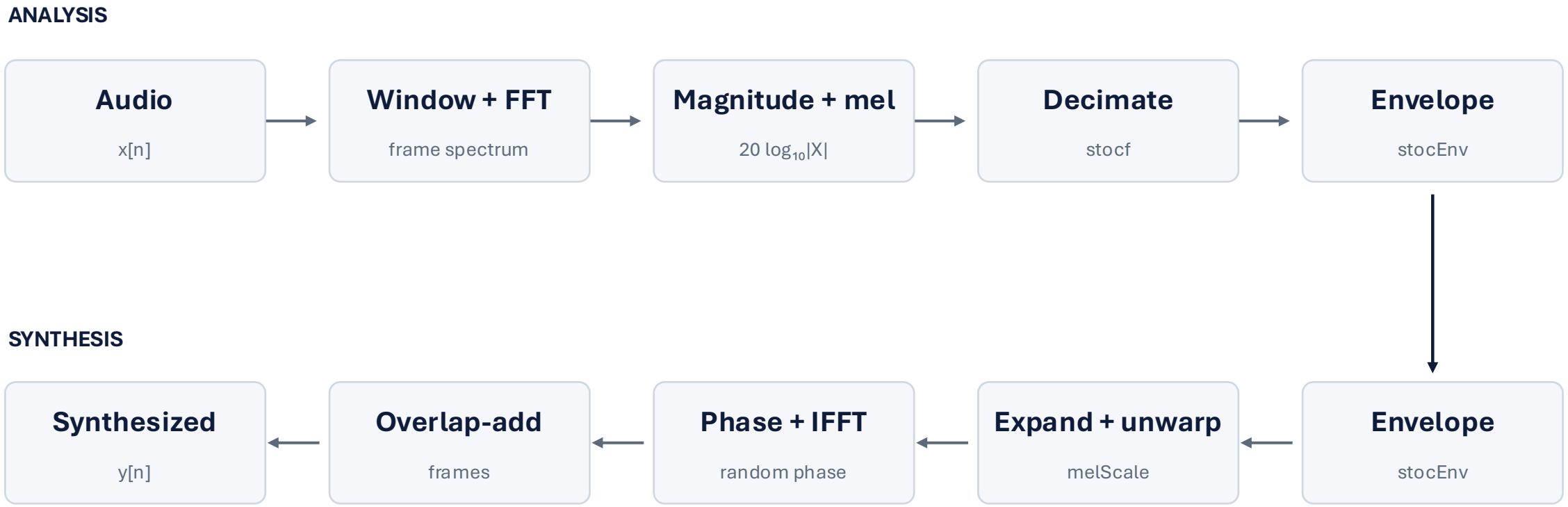
$$= |\tilde{X}_l[k]|\, e^{j\angle U[k]}$$

$|\tilde{X}_l[k]|$: approximate magnitude spectrum of x[n] $\angle U[k]$: random noise phases

l: frame number

Stochastic model system

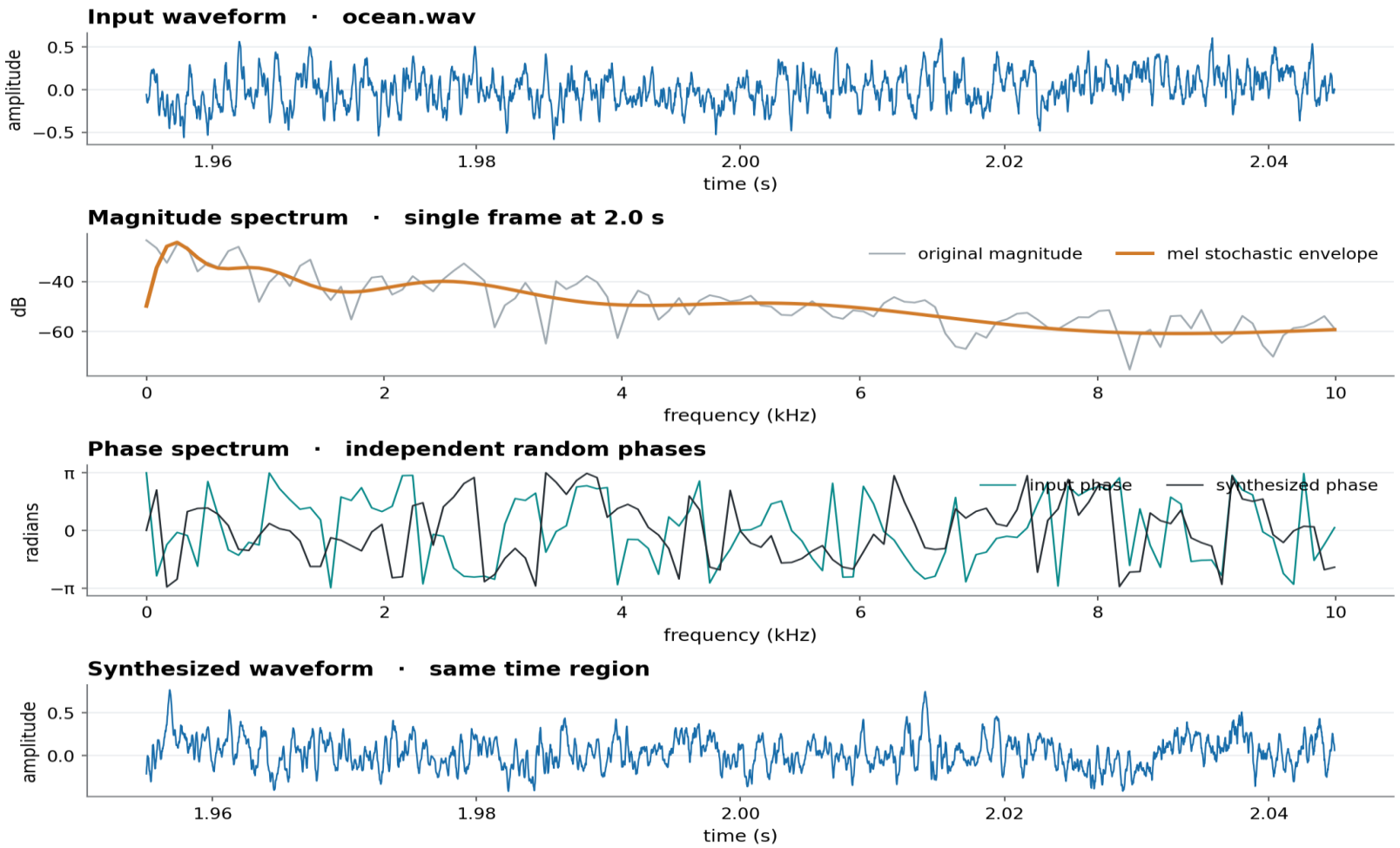
From a waveform to a compact envelope, and back to a new waveform



melScale = 1: nonuniform Hz resolution • stocf: amount of envelope compression

Example: stochastic approximation of a noise-like sound

ocean.wav · input frame, mel envelope, random phase and synthesized output (N = 512)



What to inspect

- broad magnitude envelope
- randomized phases
- similarity of texture after synthesis
- differences in exact waveform shape

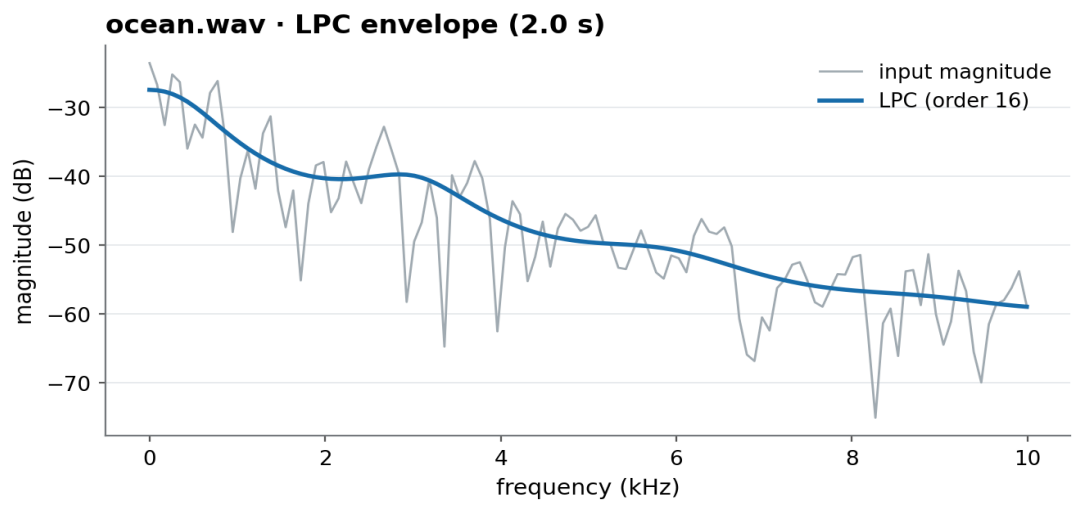
Envelope approximation

ocean.wav, frame at 2.0 s · LPC envelope vs uniform- and mel-frequency smoothing

LPC approximation

$$\hat{x}[n] = \sum_{k=1}^{K} a_k \, x[n-k]$$

$$E = \sum_{n=K}^{N-1} (x[n] - \hat{x}[n])^2$$

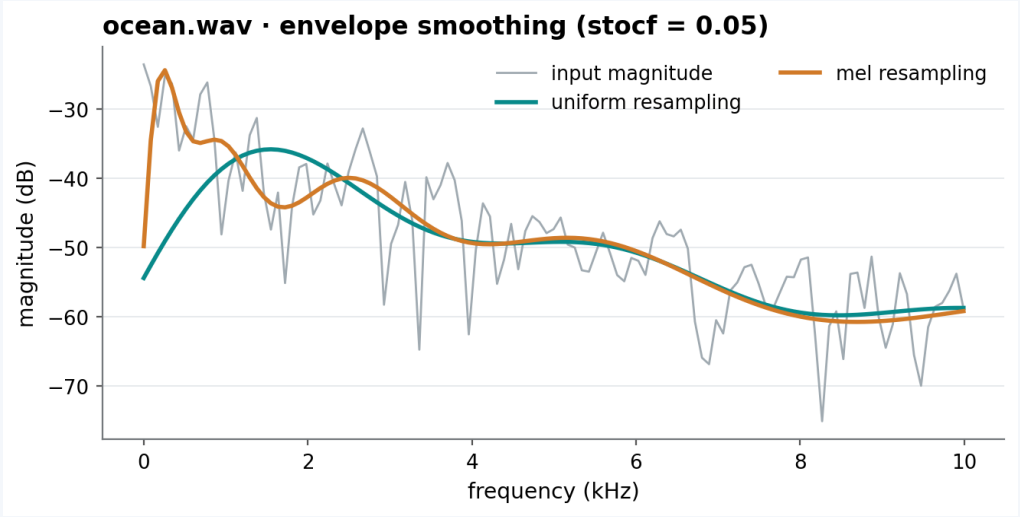


LPC order 16 · stochastic envelopes: stocf = 0.05

Envelope smoothing

$$\tilde{a}[k] = \mathrm{IDFT}(\mathrm{LP}(\mathrm{DFT}(a[k])))$$

$$b[k] = \mathrm{IDFT}(\mathrm{ZP}(\mathrm{DFT}(\tilde{a}[k])))$$



Mel-scale approximation: why change frequency axis?

Perceptual frequency spacing is nonlinear: finer resolution at low frequencies, coarser at high frequencies.

Mel frequency

$$m(f) = 2595 \log_{10}(1 + f/700)$$

f in Hz; m(f) in mels.

Equal steps on the mel axis correspond to:

- small Hz intervals at low frequencies
- large Hz intervals at high frequencies

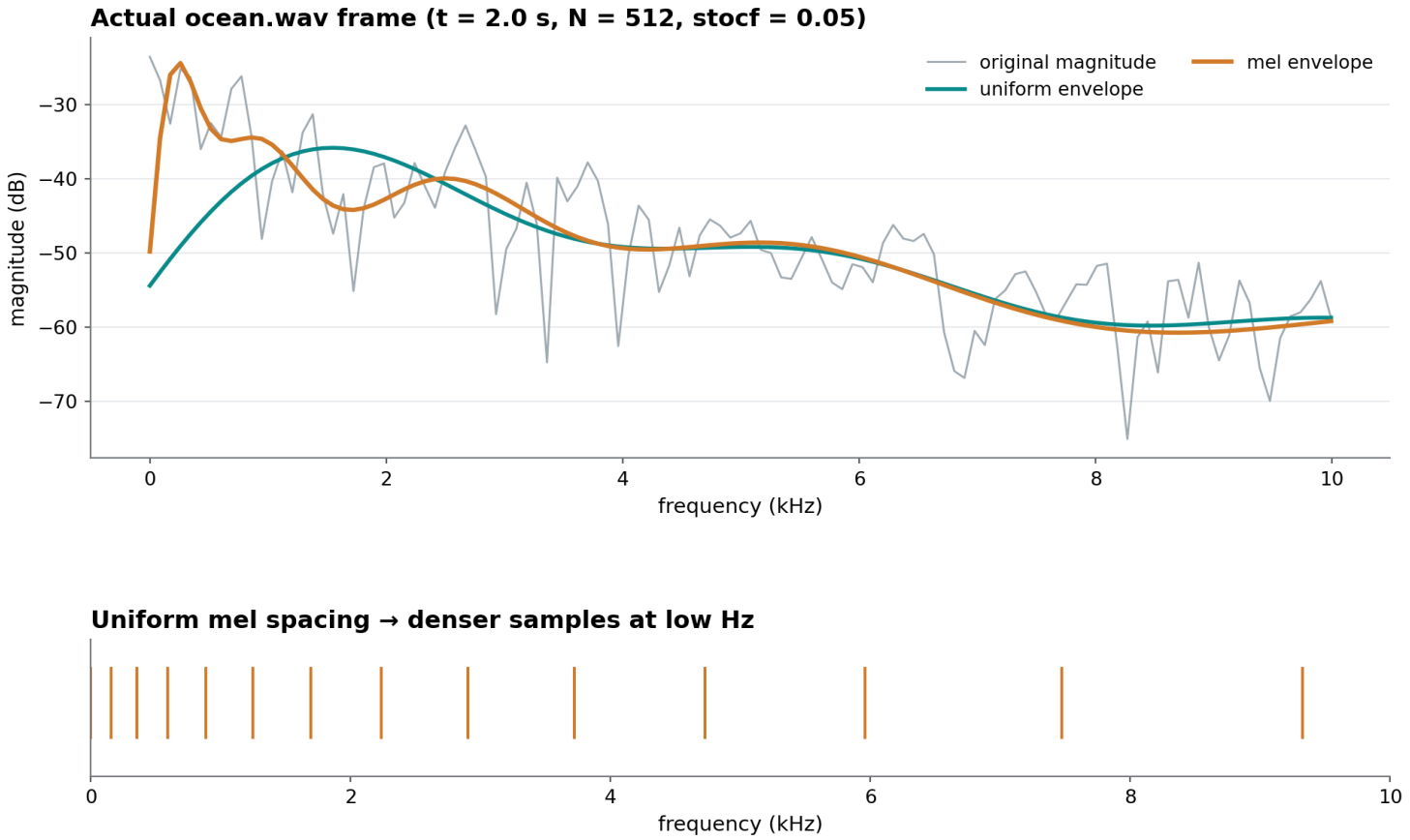
Why use it for a stochastic envelope?

With a limited number of envelope values, retain relatively more detail in the lower-frequency region.

In sms-tools, melScale=1 changes the sampling grid before resampling. It is NOT a mel-filterbank or a mel spectrogram.

Mel envelope in stochasticModel.py: interpolation + resampling

The analysis and synthesis functions use the inverse remapping of the same mel-frequency grid.



Analysis

1. FFT magnitude in dB

2. Interpolate from FFT bins to uniformly spaced mel points

3. resample(..., stocf × hN)

Synthesis

Upsample → interpolate back to FFT bins → random phase → IFFT / overlap-add

ocean.wav · frame at 2.0 s · N = 512 · stocf = 0.05 · spline interpolation + resampling

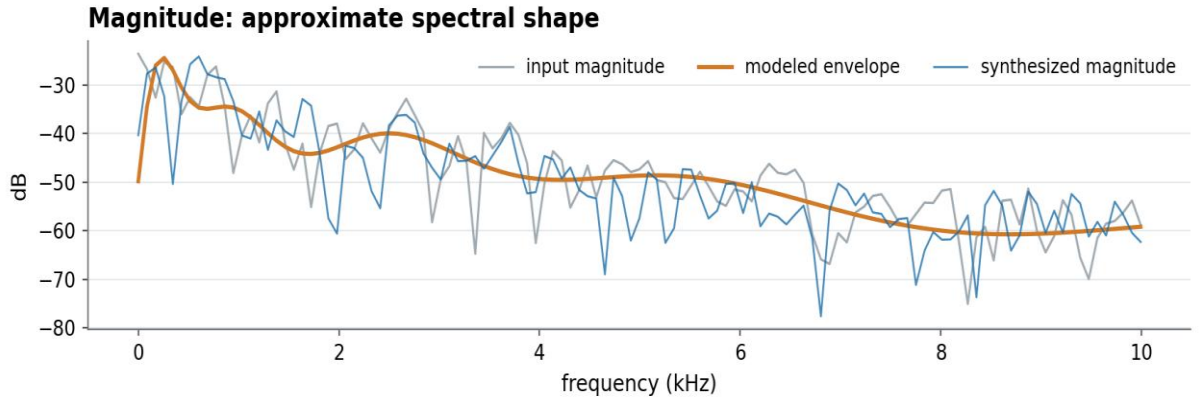
Source: https://github.com/MTG/sms-tools/blob/master/smstools/models/stochasticModel.py

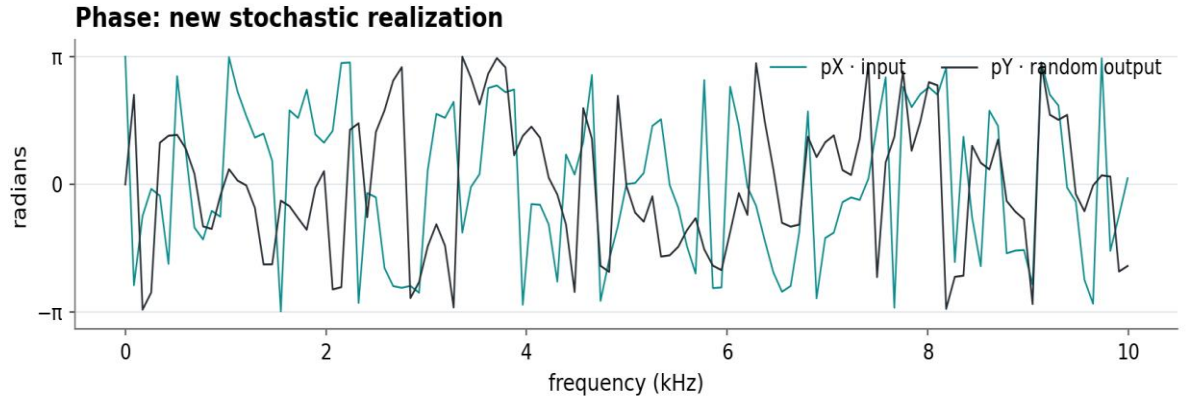
Stochastic synthesis using spectral envelopes

The synthesis keeps the magnitude envelope and generates new random phases.

$$y_{st}[n] = \mathrm{IDFT}(\,|\tilde{X}[k]|\, e^{j\varphi[k]}\,)$$

- Estimate one envelope per frame.
- Generate random phases.
- Transform each frame back to time.
- Use overlap-add to reconstruct the full signal.

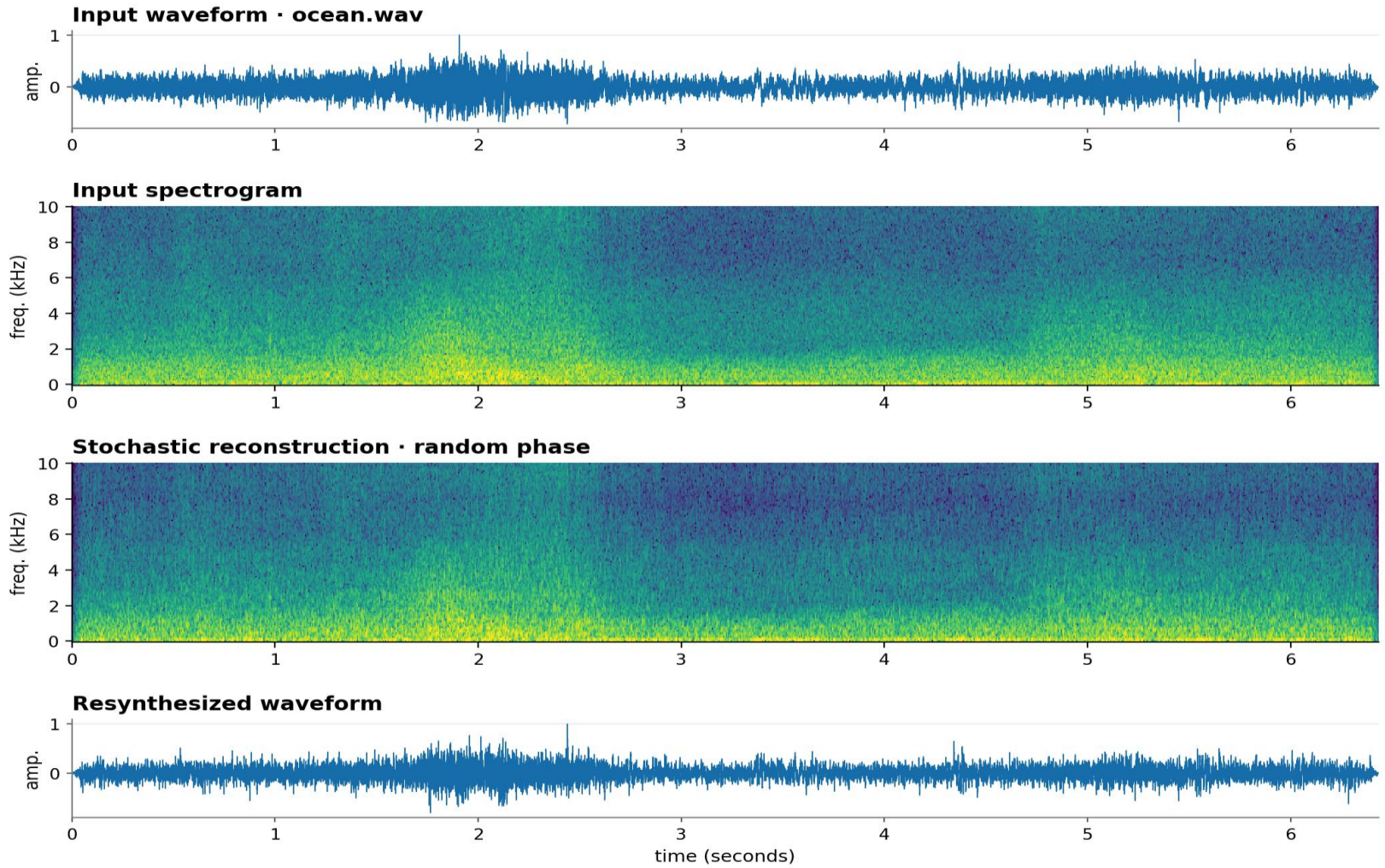




ocean.wav · same frame at 2.0 s · magnitude modeled, phase regenerated

Example: stochastic model over time

ocean.wav · compare the original waveform/spectrogram with stochastic synthesis (N = 512, H = 256)



Listening task

Decide whether the output preserves the noisy character, overall envelope, and spectral distribution of the input.

Code: stochasticModel.py

The model preserves a spectral envelope, not an exact sample sequence.

Analysis

```
stocEnv = stochasticModelAnal(
    x, H, N, stocf, fs, melScale)

# for each frame:
mX = 20 * log10(abs(FFT(xw)))
env = resample(mX, env_size)
```

Synthesis

```
y = stochasticModelSynth(
    stocEnv, H, N, fs, melScale)

# each stochastic frame:
phase = random(0, 2π)
Y = magnitude * exp(1j*phase)
y += overlap_add(real(IFFT(Y)))
```

WHAT THE CODE IMPLEMENTS

stocf controls the number of envelope samples; melScale=1 performs spline interpolation onto a uniform mel grid before resampling.

Source: smstools/models/stochasticModel.py • https://github.com/MTG/sms-tools/blob/master/smstools/models/stochasticModel.py

Sinusoidal plus residual model (SpR)

Keep the deterministic sinusoids and the complex-valued residual explicitly.

Time-domain representation

$$y[n] = y_s[n] + x_r[n]$$
$$y_s[n] = \Sigma\, A_{r,l}[n] \cos(\varphi_{r,l}[n]), \quad r = 1,\ldots,R_l$$

Spectral view (for analysis frame l)

$$Y_l[k] = \Sigma\, A_{r,l}\, W[k - \hat{f}_{r,l}] + X_{r,l}[k] = Y_{s,l}[k] + X_{r,l}[k], \quad r = 1,\ldots,R_l$$

$W[k]$: spectrum of the analysis window R_l: number of sinusoidal components

$A_{r,l}$: amplitude of sinusoid $\hat{f}_{r,l}$: normalized frequency of sinusoid

$Y_{s,l}[k]$: sinusoidal component spectrum $X_{r,l}[k] = X_l[k] - Y_{s,l}[k]$

Residuals can retain attacks, breath noise, and other components not captured by the sinusoidal model.

Code: sprModel.py

Sinusoidal plus Residual: tracking + signal subtraction.

Analysis

```
tfreq, tmag, tphase =
    sineModelAnal(x, fs, w, ...)

xr = sineSubtraction(
    x, 512, H,
    tfreq, tmag, tphase, fs)
```

Synthesis

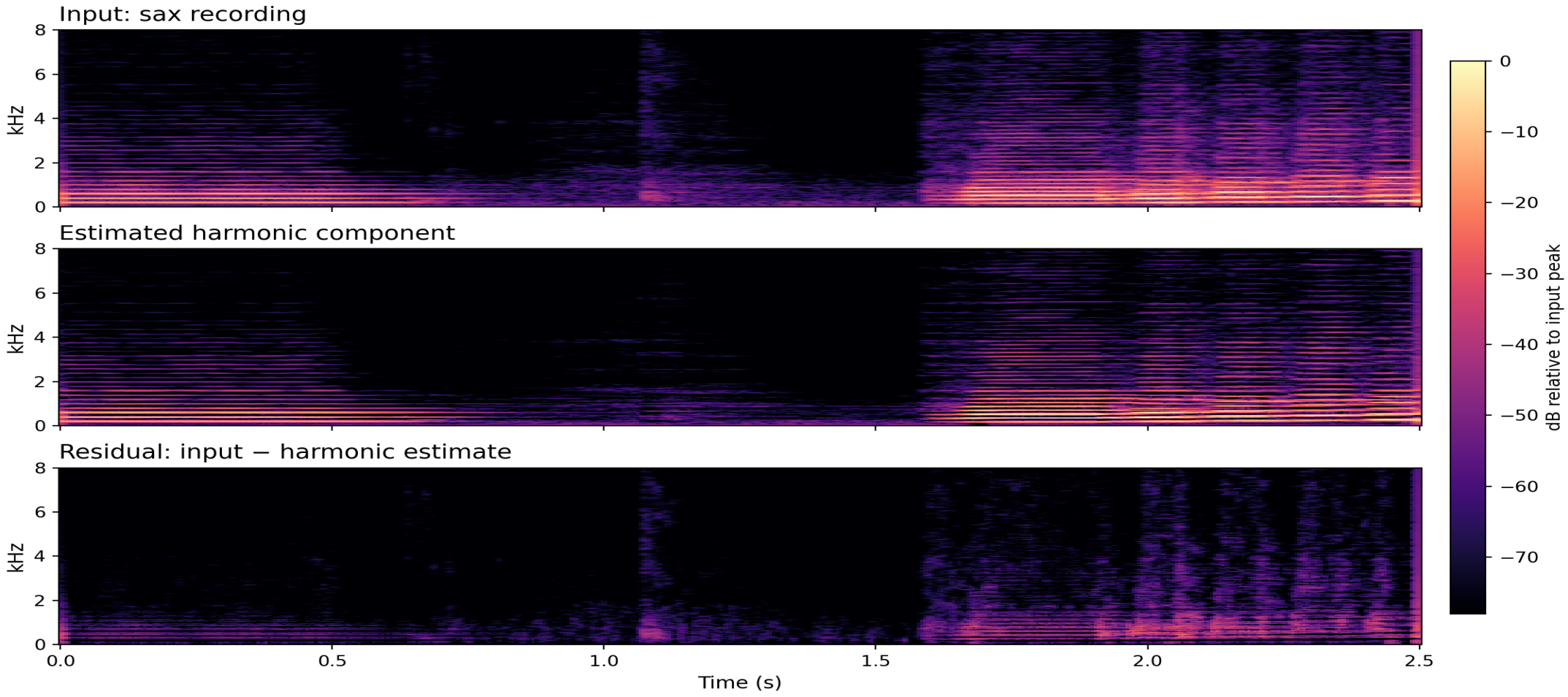
```
ys = sineModelSynth(
    tfreq, tmag, tphase,
    N, H, fs)

y = ys + xr
# preserve the explicit residual
```

Source: smstools/models/sprModel.py • https://github.com/MTG/sms-tools/blob/master/smstools/models/sprModel.py

Example: harmonic plus residual decomposition

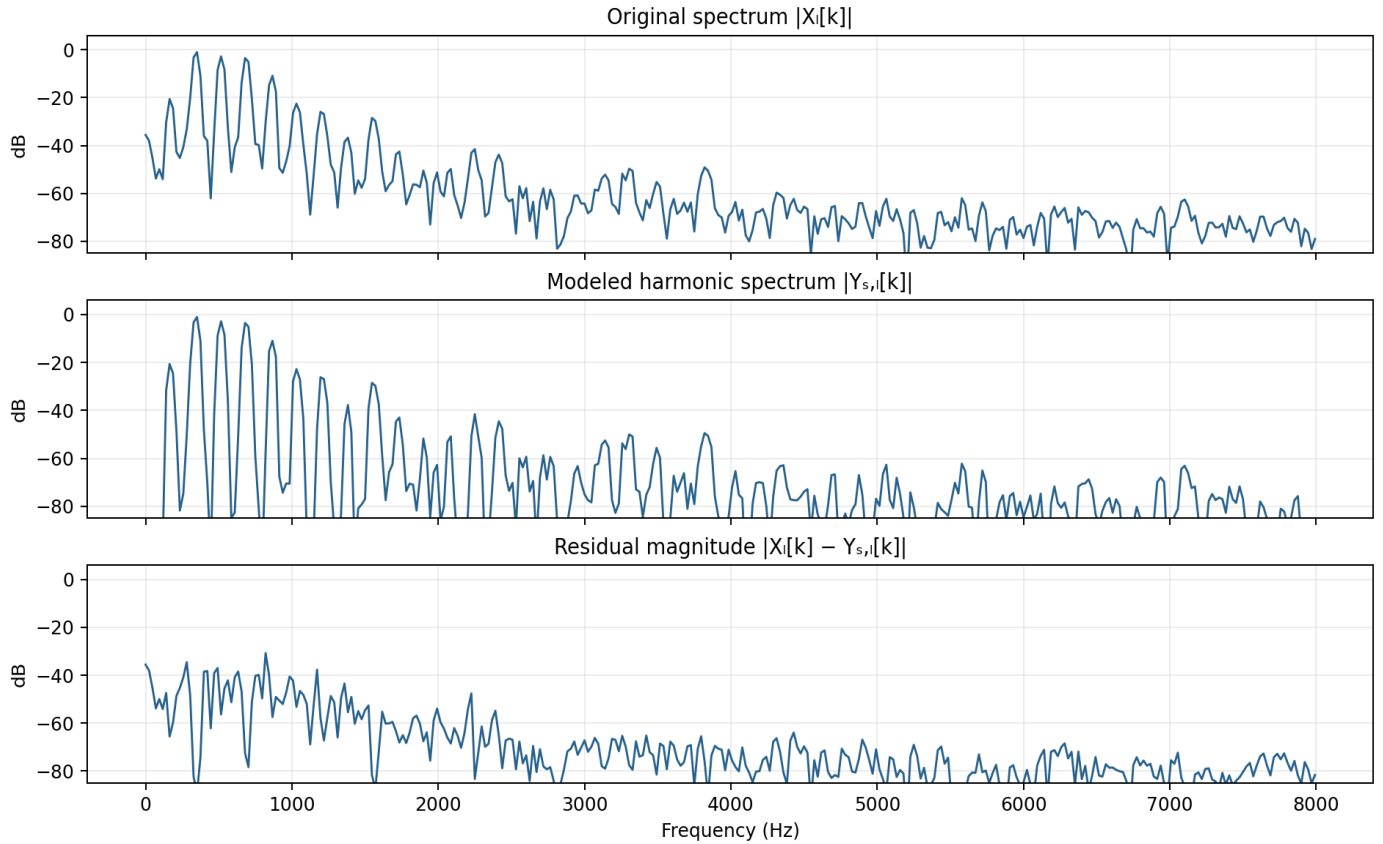
Sax example showing the input, harmonic component, and residual component.



Spectral residual subtraction (HpR)

Original, harmonic, and residual spectra for one representative sax frame.

$$X_{r,l}[k] = X_l[k] - Y_{s,l}[k]$$



How to read this slide

1. Start from the original complex spectrum $X_l[k]$.

2. Build the modeled harmonic spectrum $Y_{s,l}[k]$.

3. Subtract the modeled harmonics bin by bin.

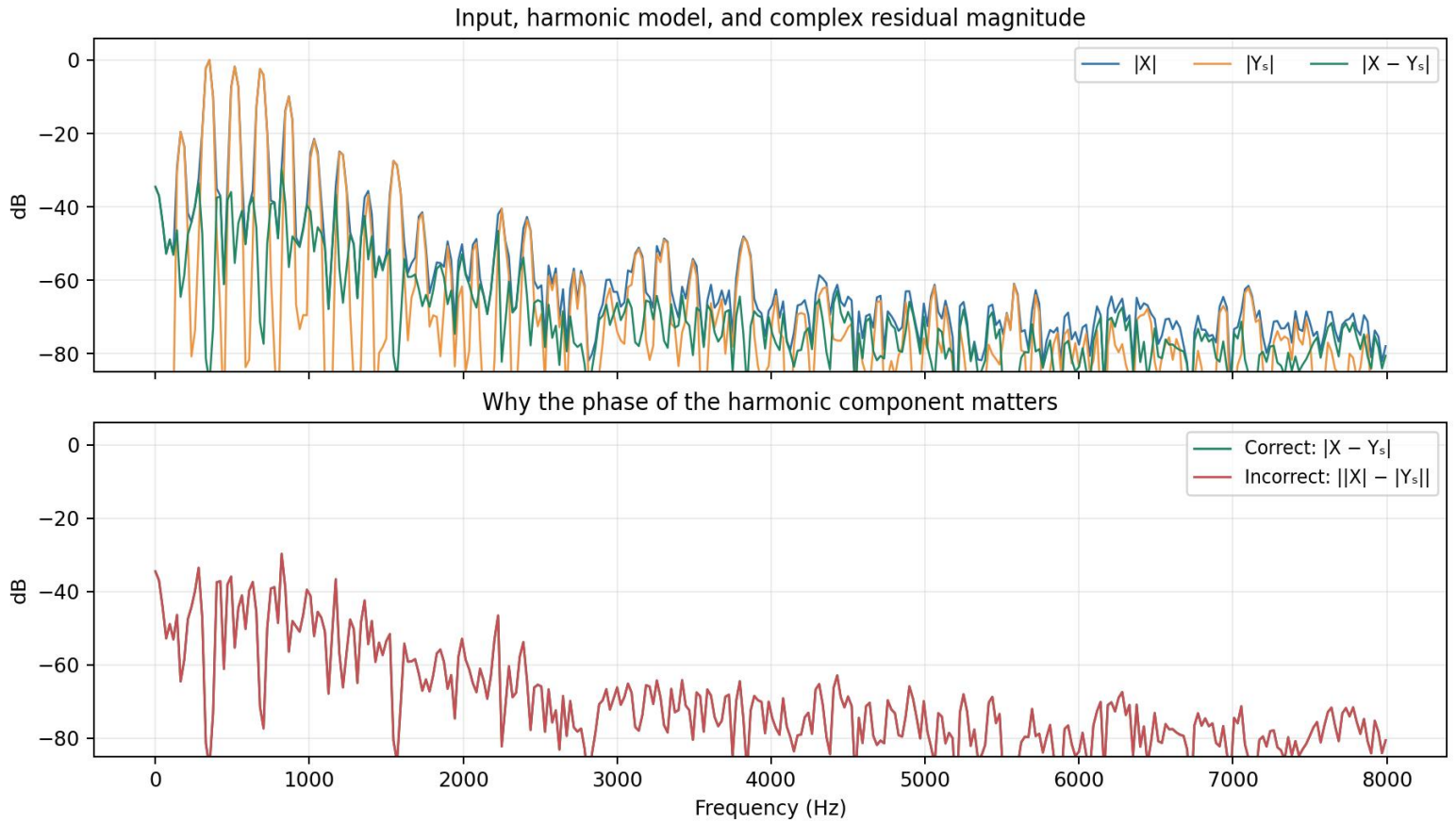
4. The remainder $X_{r,l}[k]$ is the explicit residual.

The same decomposition and sax excerpt used on slide 17.

Why subtraction must preserve phase

The residual is defined by complex subtraction, not by subtracting magnitudes only.

$$X_{r,l}[k] = X_l[k] - Y_{s,l}[k] \quad \text{and in general} \quad |X_l[k] - Y_{s,l}[k]| \neq ||X_l[k]| - |Y_{s,l}[k]||$$

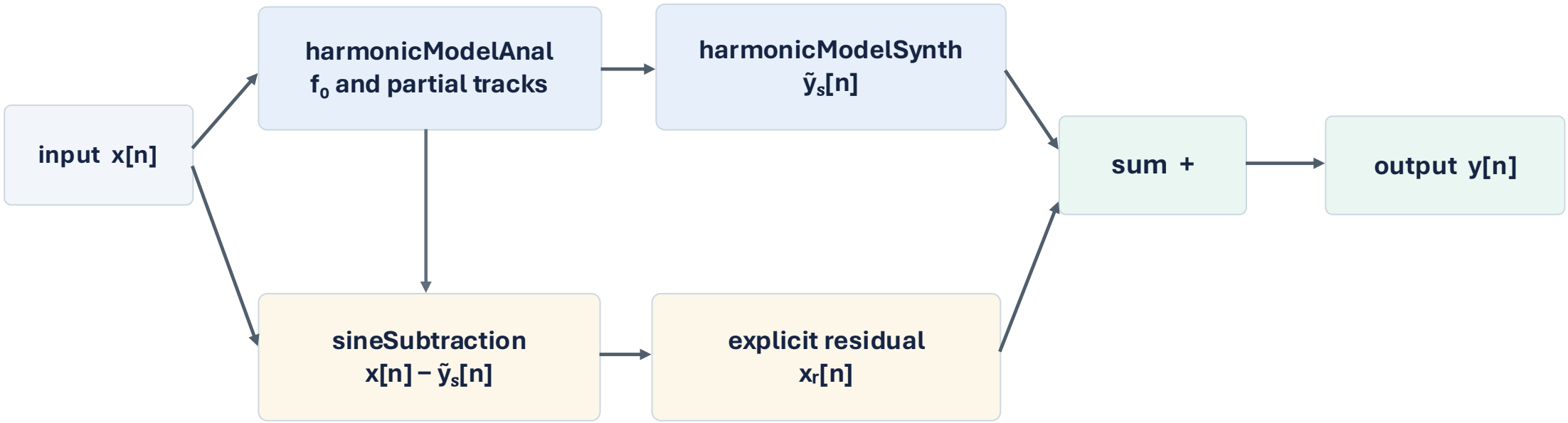


Important

- The FFT values are complex.
- Therefore the residual must be computed before taking magnitudes.
- The lower panel compares the correct residual magnitude with naive magnitude subtraction.

Harmonic plus residual system (HpR)

Harmonic tracking and residual extraction share the input; synthesis recombines both parts.

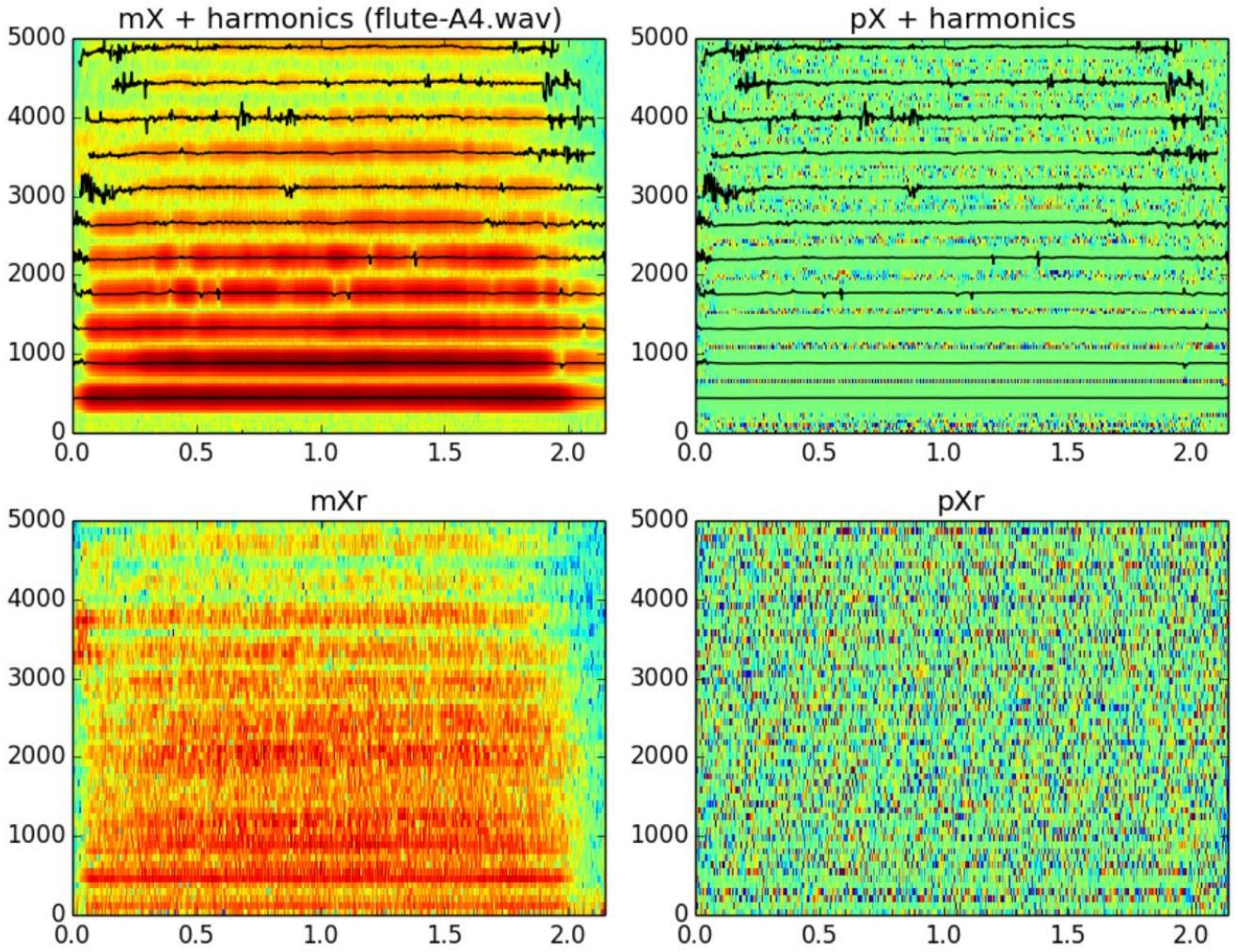


ANALYSIS: detect a fundamental and its harmonics; subtract a synthesized harmonic signal.

SYNTHESIS: recombine the explicitly stored residual with reconstructed harmonics.

Source: hprModel.py · harmonicModelAnal / utilFunctions.sineSubtraction / harmonicModelSynth

Example: harmonic plus residual decomposition



How to read it

Top: original magnitude / phase plus harmonic tracks.

Bottom: residual magnitude / phase after deterministic subtraction.

Code: hprModel.py

Analysis

```
hfreq, hmag, hphase =
    harmonicModelAnal(
      x, fs, w, N, H, ...)

xr = sineSubtraction(
    x, 512, H, hfreq,
    hmag, hphase, fs)
```

Synthesis

```
yh = sineModelSynth(
    hfreq, hmag, hphase,
    N, H, fs)

y = yh + xr
# original residual kept
```

Source: smstools/models/hprModel.py • https://github.com/MTG/sms-tools/blob/master/smstools/models/hprModel.py

Sinusoidal / harmonic plus stochastic model

Replace the explicit residual by a stochastic approximation.

$$y[n] = y_s[n] + y_{st}[n]$$

$$Y_{st,\ell}[k] = |\tilde{X}_{r,\ell}[k]|\, e^{j\varphi_\ell[k]}$$

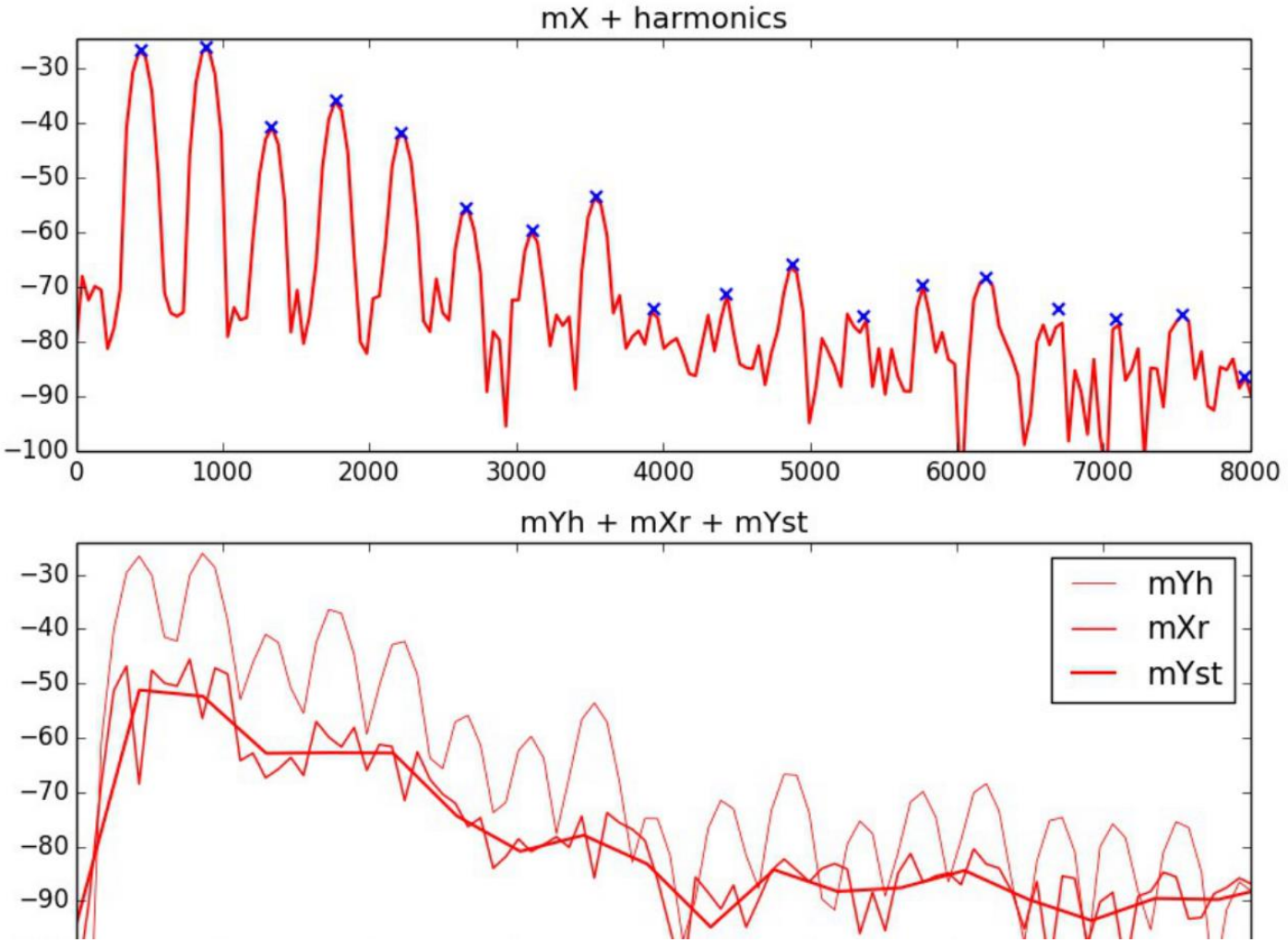
ℓ: analysis-frame index · r: residual component

- The deterministic part explains stable peaks.
- The residual magnitude is smoothed into a stochastic envelope.
- New random phases synthesize the residual texture.

HpS / SpS trade exact residual detail for a more compact perceptual approximation.

Stochastic model of the residual

The residual spectrum is smoothed and resynthesized statistically.



What changes

- Harmonic peaks remain deterministic.
- Residual detail becomes a smooth stochastic envelope.
- Local fluctuations are not preserved exactly.
- The result is usually more compact than HpR.

Code: spsModel.py

Sinusoidal plus Stochastic: represent residual by spectral envelopes.

Analysis

```
tfreq, tmag, tphase =
    sineModelAnal(x, ...)
xr = sineSubtraction(
    x, 512, H, ...)
stocEnv = stochasticModelAnal(
    xr, H, 2*H, stocf)
```

Synthesis

```
ys = sineModelSynth(
    tfreq, tmag, tphase, ...)
yst = stochasticModelSynth(
    stocEnv, H, 2*H)

y = ys + yst
```

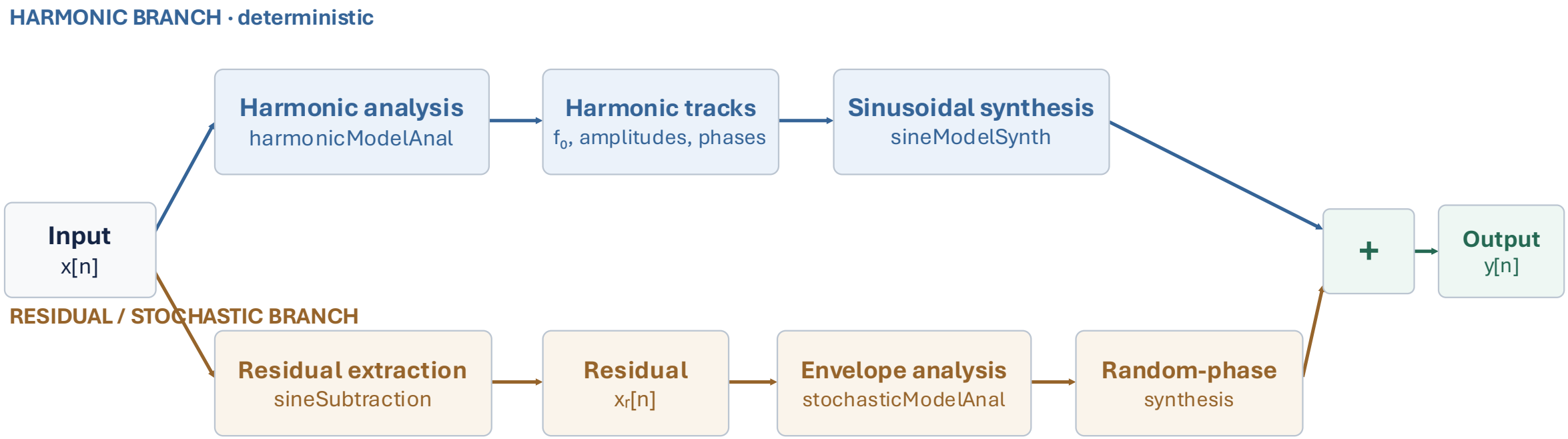
WHAT THE CODE IMPLEMENTS

SpS follows sinusoidal analysis with residual smoothing; a new noise realization replaces the explicit residual.

Source: smstools/models/spsModel.py • https://github.com/MTG/sms-tools/blob/master/smstools/models/spsModel.py

Harmonic plus stochastic system (HpS)

HpS analysis produces harmonic tracks and a residual spectral envelope; synthesis recombines harmonics and regenerated noise.



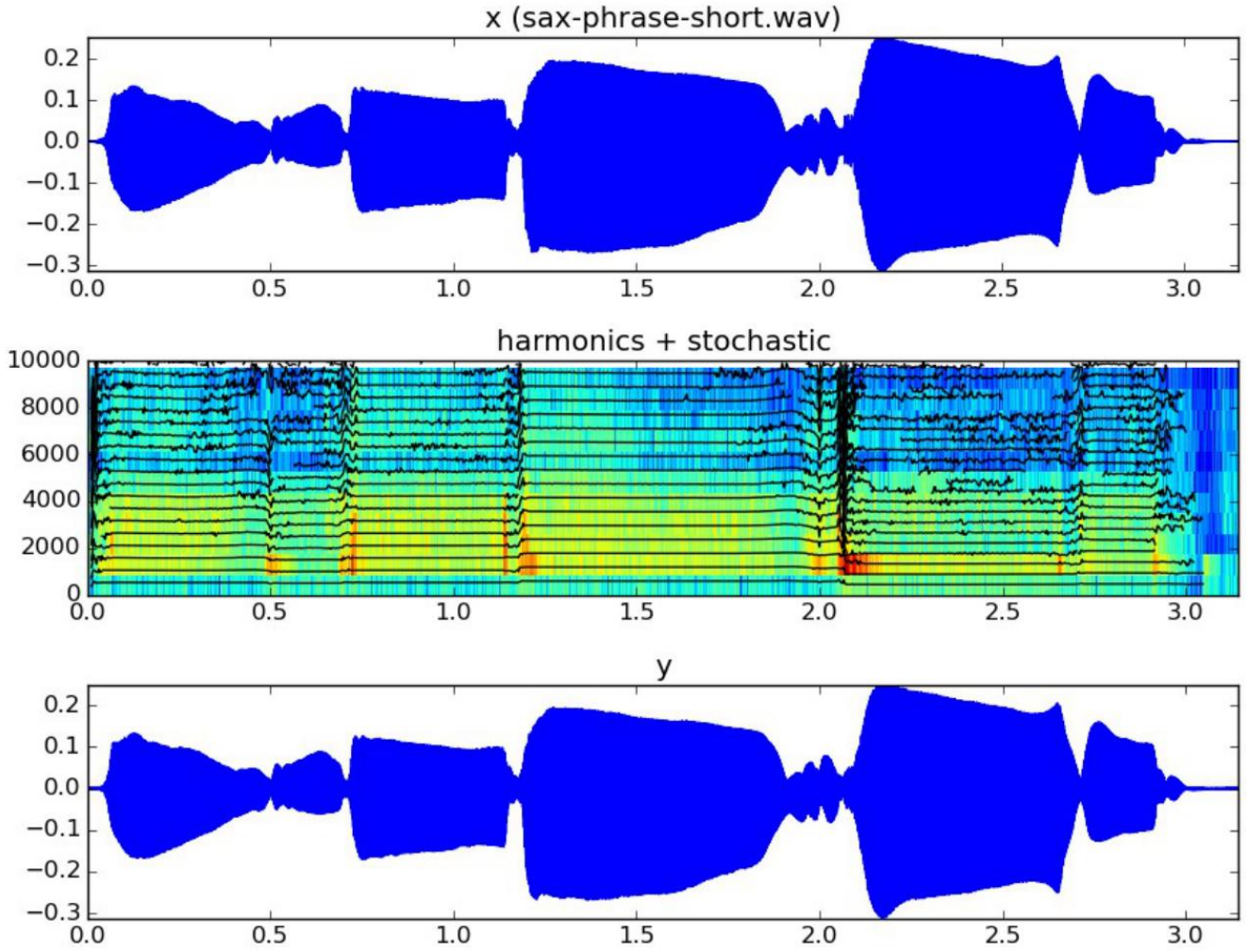
Residual extraction uses the input and its modeled harmonic component; the residual is not kept sample-for-sample.

Stochastic synthesis expands the envelope, adds new random phases, and overlaps the inverse-FFT frames.

Final output: $y[n] = y_h[n] + y_{st}[n]$

Example: harmonics + stochastic reconstruction

A sax phrase reconstructed with harmonic tracks and stochastic residual energy.



Listen for

- pitched harmonic structure
- breath/noise component
- similarity of the output envelope
- differences caused by stochastic residual synthesis

Code: hpsModel.py

Harmonic plus Stochastic: f0-based harmonics + stochastic residual.

Analysis

```
hfreq, hmag, hphase =
    harmonicModelAnal(x, ...)
xr = sineSubtraction(
    x, Ns, H, ...)
stocEnv = stochasticModelAnal(
    xr, H, 2*H, stocf)
```

Synthesis

```
yh = sineModelSynth(
    hfreq, hmag, hphase, ...)
yst = stochasticModelSynth(
    stocEnv, H, 2*H)

y = yh + yst
```

WHAT THE CODE IMPLEMENTS

The envelope retains the changing noise spectrum, while the harmonic tracks preserve pitched structure.

Source: smstools/models/hpsModel.py • https://github.com/MTG/sms-tools/blob/master/smstools/models/hpsModel.py

Choosing the right model

Model	Best for	Representation
Stochastic	Noise-like or texture-like sounds	smooth spectral envelope + random phase
SpR / HpR	Highest reconstruction fidelity	deterministic part + explicit residual
SpS / HpS	Compact perceptual approximation	deterministic part + stochastic residual envelope
Sinusoidal / Harmonic only	Clean tonal or pitched material	deterministic peaks and tracks

Exercise 7: workflow

- Choose one mostly noisy sound and one pitched sound with clear harmonic structure.
- Run a stochastic-model analysis/synthesis on the noisy sound and compare spectral envelope and output audio.
- Run a harmonic-plus-residual model on the pitched sound and inspect harmonic tracks, residual magnitude, and reconstruction.
- Run a harmonic-plus-stochastic model and compare it with the residual version.
- Write short comments that connect parameters, plots, and listening results.

Exercise 7: parameter experiments

Change one parameter at a time and explain the visible and audible consequence.

Parameter	What to observe
analysis window	resolution, smoothing, transient detail
threshold	which deterministic peaks are kept before residual subtraction
envelope smoothing	how detailed or smooth the stochastic residual becomes
stochastic scaling	how much noisy component is present in the output
number of harmonics	which deterministic components remain outside the residual

What to remember from Topic 7

Stochastic

Match a smooth magnitude envelope and use random phases.

Residual

Keep what the deterministic model does not explain.

SpR / HpR

Use explicit residuals when fidelity matters.

SpS / HpS

Use stochastic residuals when compact perceptual approximation is enough.

Validation

Always compare plots and audio together.

References and credits

SOFTWARE AND EXAMPLES

sms-tools: stochasticModel.py, sprModel.py, hprModel.py, spsModel.py, hpsModel.py

https://github.com/MTG/sms-tools/tree/master/smstools/models

sms-tools-materials: example notebooks, exercises, lectures and audio recordings

https://github.com/MTG/sms-tools-materials

Slides released under CC Attribution-Non Commercial-Share Alike license and code under Affero GPL license.